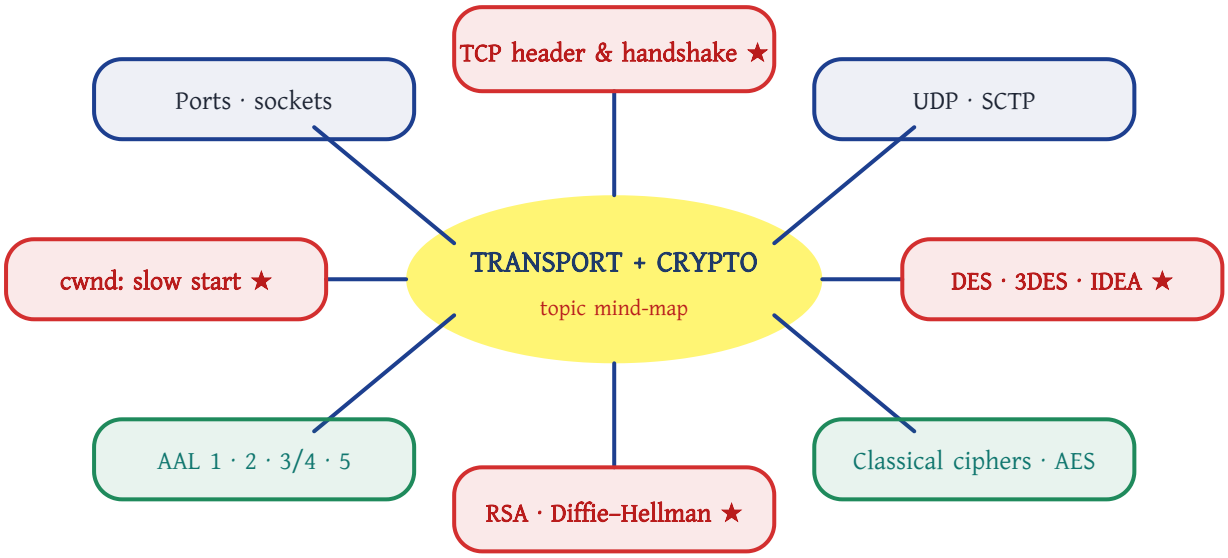
Transport Layer & Cryptography

TCP · UDP · cwnd · AAL · DES · IDEA · AES · RSA · DH: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = supporting

Syllabus coverage

"**Transport protocols**; Transmission control protocol: **connection management, transmission policy, congestion control, timer management**; UDP; **AAL of ATM**; Network security: **Cryptography, DES, IDEA, public key algorithm**". Authentication and digital signatures follow in the next lesson; the ATM cell and network layer are in the Firewalls/IP/ATM lesson.

High-yield list

① TCP 20–60 B · UDP 8 B
② SYN, SYN+ACK, ACK
③ slow start ×2 · AIMD
④ 3 dup ACKs
⑤ DES 64/56/16
⑥ IDEA 64/128/8.5
⑦ RSA: $e \cdot d \equiv 1 \bmod \varphi$

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Transport layer, ports, TCP header ..... p. 2
✓ Connection setup, release, states ..... p. 3
✓ Flow control, transmission policy, timers ... p. 4
✓ Congestion control + cwnd trace ..... p. 5
✓ UDP, TCP vs UDP, SCTP ..... p. 6
✓ AAL of ATM, cryptography basics ..... p. 7
✓ DES, 3DES, IDEA, AES ..... p. 8
✓ Public key: RSA, Diffie–Hellman ..... p. 9
✓ Night-before sheet ..... p. 10
✓ Practice MCQs + written ..... p. 11

1 The Transport Layer

> **Definition:** Layer 4 (OSI) gives **process-to-process (end-to-end)** delivery; the network layer (IP) gives only host-to-host delivery. Services: multiplexing/demultiplexing by **port**, segmentation and reassembly, and (in TCP) reliability, ordering, flow control and congestion control. It runs only in the **end hosts**, never in routers.

Port = 16-bit number → **0–65,535**

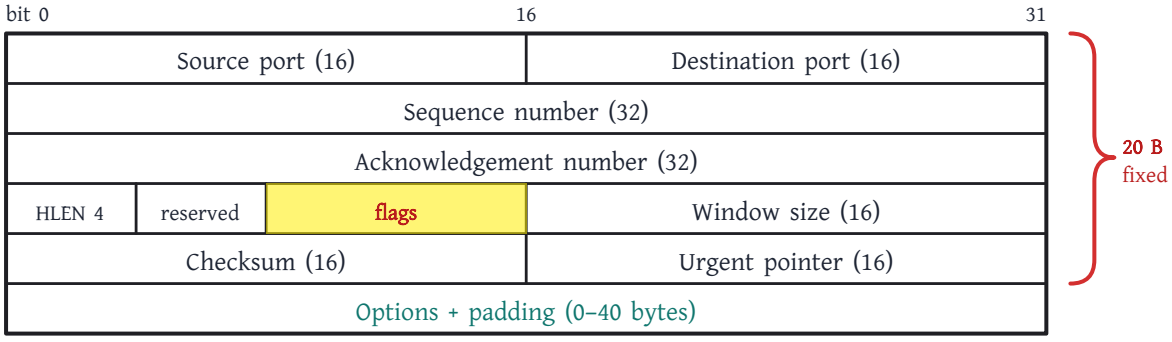
- **Well-known** 0–1023 (servers)
- Registered 1024–49,151
- Dynamic / ephemeral 49,152–65,535 (clients)

Socket = IP address + port. A TCP connection is identified by the **4-tuple** (src IP, src port, dst IP, dst port).

PDU: TCP **segment** · UDP **datagram**. IP protocol no. **TCP = 6, UDP = 17**

Service	Port	Over
FTP data · control	20 · 21	TCP
SSH · Telnet	22 · 23	TCP
SMTP	25	TCP
DNS	53	UDP/TCP
DHCP · TFTP	67/68 · 69	UDP
HTTP · HTTPS	80 · 443	TCP
POP3 · IMAP	110 · 143	TCP
SNMP · trap	161 · 162	UDP

2 TCP Segment Header



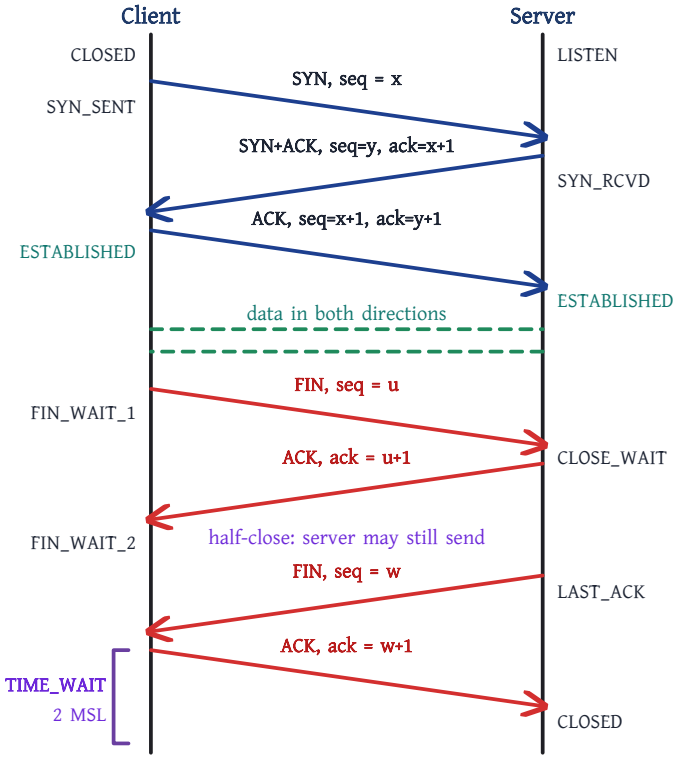
flags: URG · ACK · PSH · RST · SYN · FIN (+ ECN bits CWR, ECE)

- **HLEN** counts **32-bit words**: 5 → 20 B, 15 → **60 B**
- **Seq no.** numbers **bytes**; the ISN is random
- **Ack no.** = next byte expected (cumulative)
- **Window** = receiver's free buffer, ≤ 65,535 B unless scaled
- **Checksum**: mandatory, covers a pseudo-header

> **Memory trick: flags:** "**U**nskilled **A**ttackers **P**ester **R**eal **S**ecurity **F**olks" = URG ACK PSH RST SYN FIN, in header order.

> **Exam trap:** HLEN = 8 means 8 × 4 = **32 bytes** of header, not 8.

3 TCP Connection Management



3-way open (blue) · 4-way close (red)

3-way handshake: both sides choose and confirm their **initial sequence numbers**. SYN and FIN each **consume one sequence number**; a pure ACK consumes none.

4-way close: TCP is **full duplex**, so each direction is closed separately (FIN + ACK each way). Server ACK and FIN may be combined → 3 segments.

States: TCP has **11 states**. The **active closer** waits in **TIME_WAIT for 2 MSL** so a lost final ACK can be resent and old duplicates die out.

State	Meaning	State	Meaning
LISTEN	server waits for a SYN (passive open)	CLOSE_WAIT	got peer's FIN; app not closed yet
SYN_SENT	client sent SYN (active open)	LAST_ACK	sent own FIN, awaits its ACK
SYN_RCVD	half-open: SYN+ACK sent	CLOSING	both sides sent FIN at once

Why three segments, not two? Each side must have its own ISN **acknowledged**; a two-way exchange would let an old, delayed SYN open a ghost connection.

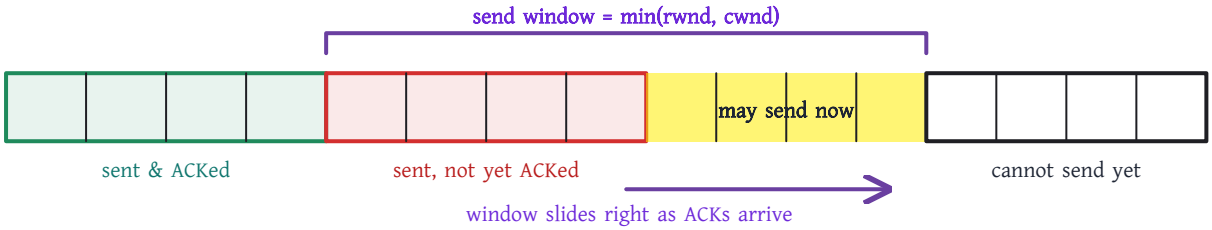
Worked example ✍️: Client ISN = 1000, server ISN = 5000. ① SYN seq = 1000 ② SYN+ACK seq = 5000, ack = **1001** ③ ACK seq = 1001, ack = 5001. Client now sends 400 bytes: seq = 1001 (bytes 1001–1400); server replies ack = **1401**.

Exam trap: **RST** aborts a connection at once (no 4-way exchange). **SYN flood**: the attacker never sends the final ACK, filling the server's half-open (SYN_RCVD) queue; defence = **SYN cookies**.

Memory trick: Open = "**knock, knock-back, enter**". Close = two **goodbyes and two nods**. Whoever says goodbye first waits at the door (TIME_WAIT).

4 Flow Control: the Sliding Window

Idea: The receiver advertises **rwnd** (free buffer) in every segment; the sender keeps unacknowledged bytes ≤ window. Effective window = **min(rwnd, cwnd)**: flow control protects the **receiver**, congestion control protects the **network**.



byte stream seen by the sender

Worked example ✍️: Max throughput ≈ window / RTT. Window 65,535 B, RTT 100 ms → 65,535 × 8 / 0.1 ≈ **5.24 Mbps**, whatever the link speed. That is why the **window-scale** option (shift up to 14, window up to 2^{30} B ≈ 1 GB) exists for long fat networks.

Transmission policy (efficiency)

Problem / mechanism	What it does
Silly window syndrome	tiny segments waste bandwidth (40 B header for 1 B data)
Nagle's algorithm (sender)	hold small data until the outstanding data is ACKed or an MSS fills
Clark's solution (receiver)	advertise 0 until min(MSS, half the buffer) is free
Delayed ACK	wait (≤ 500 ms, typically 200 ms) to piggyback or combine ACKs
Zero window	sender stops; probes with the persist timer

5 TCP Timer Management

- **Retransmission (RTO):** resend if no ACK in time
- **Persist:** breaks the zero-window **deadlock** with window probes
- **Keepalive:** checks an idle peer (default 2 hours)
- **TIME_WAIT:** 2 × MSL after an active close

RTO: Jacobson / Karn:

SRTT = ⅞ SRTT + ⅛ R

RTTVAR = ¾ RTTVAR + ¼ |SRTT − R|

RTO = SRTT + **4 × RTTVAR**

Karn: no RTT sample from a retransmitted segment; **double** RTO on every timeout

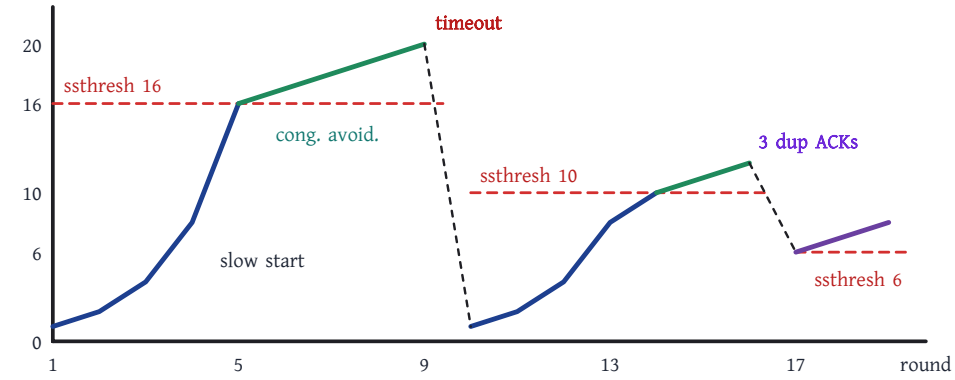
Worked example ✍️: SRTT = 100 ms, RTTVAR = 20 ms, new sample R = 120 ms. RTTVAR = 15 + ¼(20) = 20 ms; SRTT = 87.5 + 15 = 102.5 ms; RTO = 102.5 + 80 = **182.5 ms** (RFC 6298 then applies a 1 s floor).

6 TCP Congestion Control

Idea: The sender keeps a **congestion window (cwnd)**, counted in MSS, and a threshold **ssthresh**. Loss is taken as the sign of congestion. Overall rule: **AIMD**: additive increase, multiplicative decrease.

Phase / event	Rule	Growth
Slow start (cwnd < ssthresh)	+1 MSS per ACK	×2 every RTT
Congestion avoidance (≥ ssthresh)	+1 MSS per RTT	linear
Timeout (Tahoe, Reno)	ssthresh = cwnd/2; cwnd = **1 MSS**	slow start again
3 dup ACKs: fast retransmit	resend the lost segment now, no RTO wait	—
… then **fast recovery** (Reno)	ssthresh = cwnd/2; cwnd = ssthresh	linear
… Tahoe instead	ssthresh = cwnd/2; cwnd = 1	slow start

Worked trace: ssthresh = 16 at the start



cwnd (MSS) vs transmission round · Reno

Round	1	2	3	4	5	6	7	8	9
cwnd	1	2	4	8	16	17	18	19	20
Round	10	11	12	13	14	15	16	17	18
cwnd	1	2	4	8	10	11	12	6	7

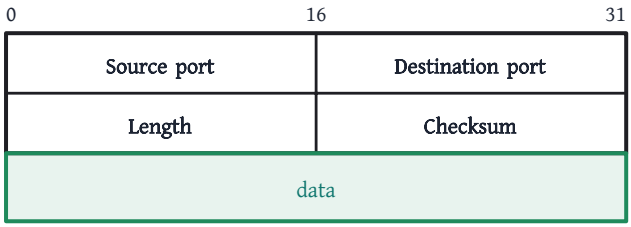
Timeout at 20 → ssthresh = **10**, cwnd = 1. Slow start stops at ssthresh (8 → 10, not 16). Three duplicate ACKs at 12 → ssthresh = cwnd = **6** (Reno skips slow start).

Memory trick: "**Timeout = start over from 1; dup ACKs = cut in half.**" 3 dup ACKs show later segments still get through.

Exam trap: "Slow" start is actually **exponential**. It is slow only because cwnd starts at 1 MSS (modern stacks start at up to 10). Tahoe 1988, Reno 1990; CUBIC is the Linux default.

7 UDP: User Datagram Protocol

Definition: UDP (RFC 768, 1980) is a **connectionless, unreliable** transport: no handshake, no ACKs, no retransmission, no ordering, no flow or congestion control. It adds only ports and a checksum to IP, and **preserves message boundaries**.



4 fields × 16 bits = 8 bytes

- Header fixed at **8 bytes** (TCP: 20–60)
- **Length** = header + data, minimum 8
- Checksum **optional in IPv4** (0 = unused), **mandatory in IPv6**
- Max payload over IPv4: 65,535 − 20 − 8 = **65,507 B**
- Only UDP supports **broadcast and multicast**

Used by: DNS queries, DHCP, TFTP, SNMP, RIP (520), NTP (123), VoIP/RTP, live streaming, online games, and **QUIC** (HTTP/3 over UDP 443).

8 TCP vs UDP

Feature	TCP	UDP
Connection	connection-oriented (handshake)	connectionless
Reliability	ACK + retransmission, in-order	best effort; may lose, reorder
Data unit	byte stream (segments)	messages (datagrams)
Header	20–60 bytes	8 bytes
Flow / congestion ctrl	yes / yes	no / no
Delivery	unicast only	unicast, broadcast, multicast
Examples	HTTP, HTTPS, FTP, SMTP, SSH	DNS, DHCP, SNMP, VoIP

Other transport protocols

SCTP: reliable and **message-oriented**, with **multi-streaming** and **multi-homing**; 4-way handshake (INIT, INIT-ACK, COOKIE-ECHO, COOKIE-ACK); used for telecom signalling (SIGTRAN).

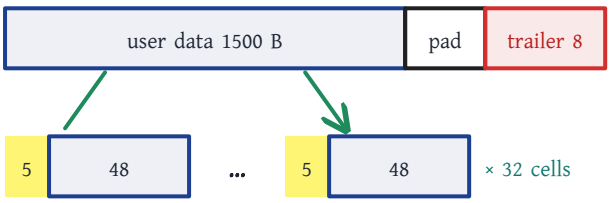
DCCP: unreliable datagrams **with** congestion control, for streaming media. **RTP** runs over UDP and adds timestamps and sequence numbers for real-time audio/video.

Exam trap: "UDP is unreliable" means it gives **no delivery guarantee**, not that it corrupts data: the checksum still detects errors. DNS uses **TCP** 53 for zone transfers and large replies.

9 ATM Adaptation Layer (AAL)

Role: An ATM cell is **53 bytes** (5 header + 48 payload). The AAL, above the ATM layer, adapts upper-layer data to cells through two sublayers: **CS** (convergence) and **SAR** (segmentation and reassembly).

AAL	Class	Traffic	Payload / cell	Typical use
AAL1	A	CBR, timed	47 B	circuit emulation, voice
AAL2	B	VBR, timed	mini-packets	compressed voice/video
AAL3/4	C, D	data (conn. / connless)	44 B	SMDS, legacy data
AAL5	C, D	data, simple (SEAL)	**48 B**	**IP over ATM**, LANE



AAL5: one frame → many cells

Worked example ✍️: 1500-byte IP packet over AAL5: 1500 + 8 = 1508 → ⌈1508/48⌉ = **32 cells** (pad 28 B). On the wire 32 × 53 = 1696 B, efficiency 1500/1696 ≈ **88.4%**. Trailer: UU, CPI, length, **CRC-32**; the PTI bit marks the last cell.

Memory trick: "**1 = Constant, 2 = Variable, 3/4 = old data, 5 = the data workhorse.**" Class letters A → D follow the AAL numbers.

10 Cryptography: Basics

Terms: **Plaintext** → (encryption with a **key**) → **ciphertext** → (decryption) → plaintext. **Cryptology** = cryptography (making codes) + cryptanalysis (breaking them). Goals: confidentiality, integrity, authentication, non-repudiation. **Kerckhoffs**: security rests on the key alone.

	Symmetric (secret key)	Asymmetric (public key)
Keys	one shared key	public + private pair
Speed	fast (bulk data)	100–1000× slower
Keys for n users	**n(n − 1)/2**	2n
Examples	DES, 3DES, IDEA, AES, RC4, Blowfish	RSA, Diffie–Hellman, ECC, ElGamal

11 Classical Ciphers

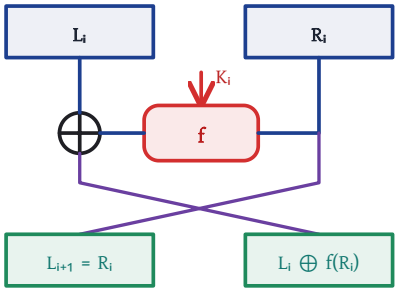
Substitution ✍️: replace each letter. **Caesar** (shift 3): ATTACK → **DWWDFN**. Monoalphabetic: 26! keys, but broken by **letter frequency**. Polyalphabetic: Vigenère, Playfair. One-time pad: perfect secrecy.

Transposition ✍️: reorder the letters. **Rail fence** (2 rails): ATTACKATDAWN → row 1 ATCADW + row 2 TAKTAN = **ATCADWTAKTAN**. Letter frequencies stay unchanged.

Shannon: a strong cipher needs **confusion** (substitution, S-boxes) and **diffusion** (permutation, P-boxes). Block ciphers (DES, AES) encrypt fixed blocks; stream ciphers (RC4, ChaCha20) encrypt bit by bit or byte by byte.

12 DES and Triple DES

- IBM (from Lucifer), FIPS 46, **1977**
- Block **64 bits**; key 64 bits, **56** effective (8 parity)
- **16 Feistel rounds**, 48-bit round keys; IP … IP^{-1}
- f: expand 32 → 48, XOR key, **8 S-boxes (6 → 4 bits)**, P-box
- S-boxes = the only **non-linear** part
- Brute-forced: EFF "Deep Crack", 1998 (56 h)



one Feistel round

3DES (EDE): $C = E_{K3}(D_{K2}(E_{K1}(P)))$. 3 keys: 168-bit key, ≈ 112-bit strength; 2 keys (K1 = K3): 112-bit key. K1 = K2 = K3 → single DES. **Double DES** ≈ 57 bits only (**meet-in-the-middle**).

13 IDEA and AES

IDEA: Lai & Massey (ETH Zürich). AES: Rijndael (Daemen & Rijmen), FIPS 197; each round = SubBytes, ShiftRows, MixColumns, AddRoundKey.

Cipher	Block	Key	Rounds	Structure / note
DES	64	56	16	Feistel
IDEA 1991	64	128	8 + ½	XOR, + mod 2^{16}, × mod $2^{16}+1$; PGP
AES 2001	128	128/192/256	10/12/14	SPN (Rijndael)

Memory trick: DES "**64-56-16**" (block, key, rounds). IDEA "**64-128-8**". AES rounds "**10-12-14**" = key/32 + 6.

Exam trap: IDEA is **not Feistel**: 6 subkeys × 8 rounds + 4 (half-round) = **52** subkeys. AES is not Feistel either.

14 Public-Key Cryptography

Idea: Each user has a **public key** (published) and a **private key** (secret). For confidentiality: encrypt with the receiver's public key, decrypt with the receiver's private key. Security rests on a hard problem: **factoring** (RSA) or the **discrete logarithm** (Diffie–Hellman, ElGamal, ECC). Diffie–Hellman (**1976**) came first, RSA (1977) next.

RSA (Rivest, Shamir, Adleman, 1977)

1. Choose primes p, q; $\mathbf{n = p \times q}$
2. $\boldsymbol{\varphi(n) = (p - 1)(q - 1)}$
3. Choose e with $1 < e < \varphi(n)$, $\gcd(e, \varphi(n)) = 1$
4. $d = e^{-1} \bmod \varphi(n)$, i.e. $\mathbf{e \cdot d \equiv 1 \pmod{\varphi(n)}}$
5. Public key (e, n) · private key (d, n)
6. Encrypt $C = M^e \bmod n$ · decrypt $M = C^d \bmod n$

Worked example ✍️: p = 3, q = 11 → n = 33, $\varphi = 2 \times 10 = 20$.

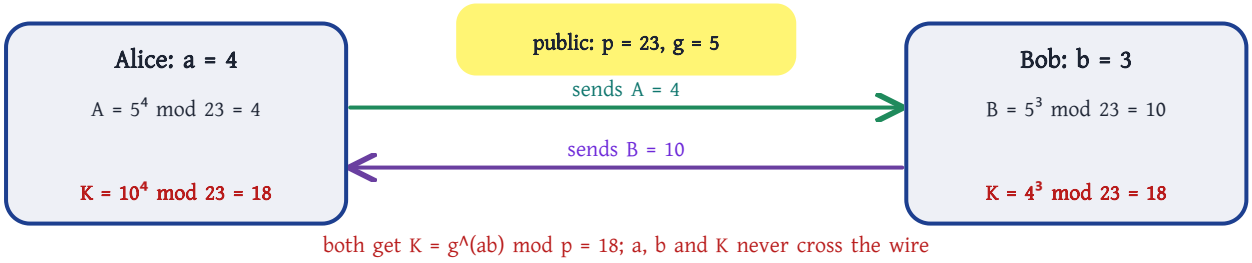
e = 7 ($\gcd(7, 20) = 1$). $7d \equiv 1 \pmod{20}$: $7 \times 3 = 21 = 20 + 1 \rightarrow \mathbf{d = 3}$.

M = 2: $C = 2^7 \bmod 33 = 128 - 99 = \mathbf{29}$.

Check: $29^3 \bmod 33$; $29 \equiv -4$, $(-4)^3 = -64 \equiv -64 + 66 = \mathbf{2}$ ✓

Real keys: n of **2048 bits** or more. RSA is slow, so it is used to exchange a session key or to sign (see the Authentication & Digital Signatures lesson), while AES encrypts the bulk data: the **hybrid** scheme used by TLS and PGP.

Diffie–Hellman key exchange



key agreement over an open channel

Check the arithmetic ✍️: $5^4 = 625 = 27 \times 23 + 4 \rightarrow A = 4$. $5^3 = 125 = 5 \times 23 + 10 \rightarrow B = 10$. Alice: $10^2 = 100 \equiv 8$, $8^2 = 64 \equiv \mathbf{18}$. Bob: $4^3 = 64 \equiv \mathbf{18}$ ✓

Exam trap: Plain Diffie–Hellman **agrees on a key; it does not encrypt** and has no authentication, so it falls to a **man-in-the-middle** attack. A and B are public values, not the key.

Memory trick: RSA: "**Public Encrypts, Private Decrypts**" for secrecy. **ECC:** 256-bit ECC ≈ 3072-bit RSA ≈ AES-128 strength, so much smaller keys.

Night-Before-the-Exam Sheet

Fact sheet:

Ports 16 bits: 0–1023 well-known

TCP = 6, UDP = 17 (IP protocol no.)

TCP header 20–60 B · HLEN × 4

UDP header 8 B · max payload 65,507 B

SYN → SYN+ACK → ACK; SYN/FIN use 1 seq no.

TIME_WAIT = 2 MSL · 11 TCP states

window = min(rwnd, cwnd) · rate ≤ W/RTT

RTO = SRTT + 4 · RTTVAR

slow start ×2 per RTT · CA +1 MSS per RTT

timeout: cwnd = 1 · 3 dup ACKs: cwnd = ½

ATM cell 53 = 5 + 48 · AAL5 trailer 8 B

symmetric keys $n(n-1)/2$ · public $2n$

DES 64/56/16 · 3DES 168 (≈112)

IDEA 64/128/8.5, 52 subkeys

AES 128 block; 10/12/14 rounds

RSA $\varphi = (p-1)(q-1)$, $ed \equiv 1 \bmod \varphi$

Top 12 one-liners for MCQs

1. Transport layer: process-to-process delivery
2. TCP is full duplex, byte-stream
3. Persist timer: zero-window deadlock
4. Nagle = sender side, Clark = receiver side
5. Fast retransmit: 3 duplicate ACKs
6. Reno adds fast recovery to Tahoe
7. AAL1 = CBR; AAL5 = IP over ATM
8. Caesar = substitution; rail fence = transposition
9. DES S-box: 6 bits in, 4 bits out
10. IDEA used in PGP
11. RSA rests on factoring n
12. DH: key exchange, open to MITM

Top traps: HLEN counts 4-byte words · ack no. = next byte expected · "slow" start is exponential · Reno: timeout → 1, dup ACKs → half · UDP checksum optional only in IPv4 · DES key 64 bits written, 56 effective · IDEA and AES are not Feistel · Double DES ≈ single DES strength · DH does not encrypt

All tricks together: URG ACK PSH RST SYN FIN = "Unskilled Attackers Pester Real Security Folks" · knock, knock-back, enter · whoever says goodbye first waits (TIME_WAIT) · timeout = start over, dup ACKs = halve · AAL 1 constant, 2 variable, 5 data · 64-56-16, 64-128-8, 10-12-14 · Public Encrypts, Private Decrypts



one HTTPS session uses the whole lesson

hybrid encryption in TLS

✓ Practice MCQs (exam pattern, 10 marks)

1. Size of the UDP header:
(a) 20 bytes (b) 8 bytes
(c) 16 bytes (d) 4 bytes

2. Which TCP flag opens a connection?
(a) FIN (b) RST
(c) PSH (d) SYN

3. After a retransmission timeout, TCP sets cwnd to:
(a) 1 MSS (b) half of cwnd
(c) ssthresh + 3 (d) unchanged

4. The AAL most used to carry IP over ATM:
(a) AAL1 (b) AAL2
(c) AAL3/4 (d) AAL5

5. DES block size and effective key length:
(a) 64 / 56 bits (b) 128 / 128 bits
(c) 64 / 128 bits (d) 56 / 64 bits

6. The key length of IDEA:
(a) 56 bits (b) 64 bits
(c) 128 bits (d) 256 bits

7. Number of rounds in AES-256:
(a) 10 (b) 12
(c) 14 (d) 16

8. In RSA with p = 3 and q = 11, φ(n) is:
(a) 33 (b) 20
(c) 22 (d) 30

9. TCP fast retransmit is triggered by:
(a) a timeout (b) 3 duplicate ACKs
(c) a zero window (d) an RST

10. Ports 0–1023 are called:
(a) well-known (b) registered
(c) dynamic (d) ephemeral

Answer key → 1-b · 2-d · 3-a · 4-d · 5-a · 6-c · 7-c · 8-b · 9-b · 10-a

Written question: Explain slow start and congestion avoidance. With ssthresh = 8 MSS and cwnd = 1 at round 1, a timeout occurs when cwnd = 12. Trace cwnd up to round 13. 8 marks

1. Slow start: cwnd += 1 MSS per ACK, so it doubles every RTT until it reaches ssthresh 2
2. Congestion avoidance: cwnd += 1 MSS per RTT (additive increase); loss → multiplicative decrease 2
3. Rounds 1–8: 1, 2, 4, **8**, 9, 10, 11, **12** → timeout 2
4. ssthresh = 12/2 = **6**, cwnd = 1. Rounds 9–13: 1, 2, 4, **6** (capped at ssthresh), 7 2

Before the exam ✓

- ✓ Can draw the TCP header with field sizes
- ✓ Can draw the handshake and close with seq/ack numbers
- ✓ Can trace cwnd through a timeout and 3 dup ACKs
- ✓ Can do an RSA and a DH example by hand

Also practise (written):
① TCP vs UDP with 2 apps each
② AAL types with uses
③ DES round structure
④ RSA: p = 5, q = 11, e = 3 → d = 27